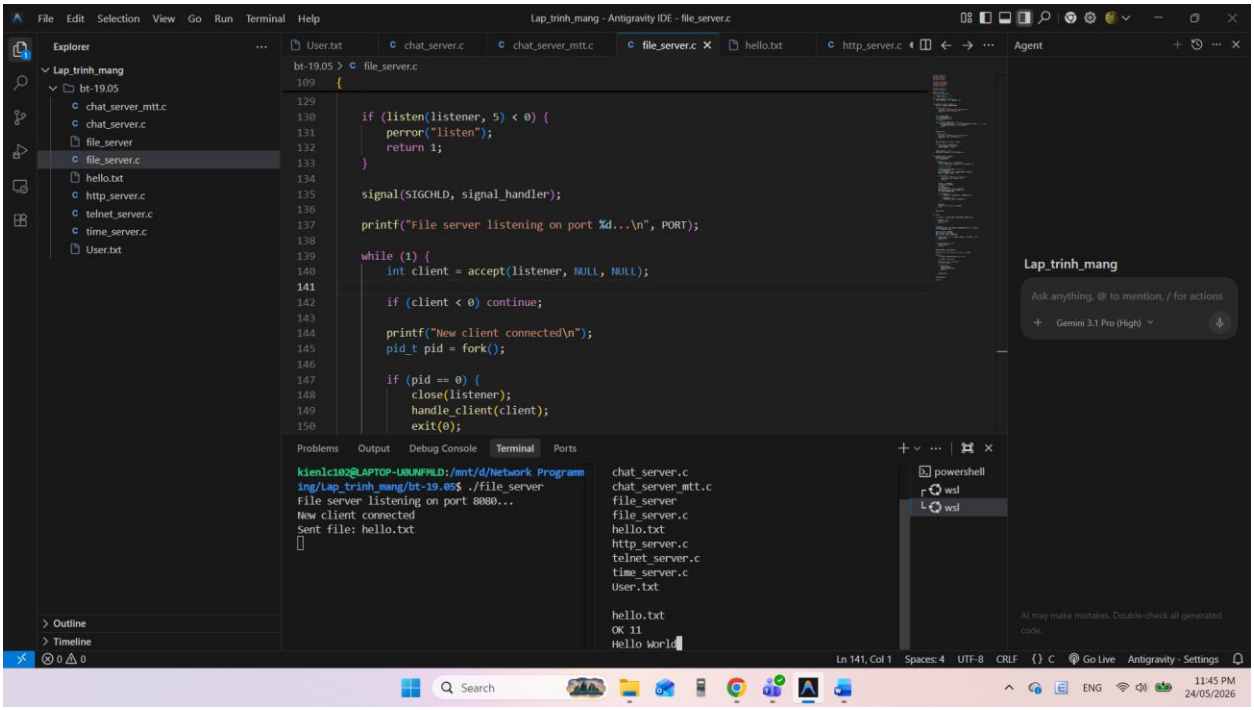
Bài 3.1 file_server:



Mã nguồn:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h>

#include <sys/socket.h>
#include <sys/types.h>
#include <arpa/inet.h>
#include <unistd.h>

#include <signal.h>
#include <sys/wait.h>

#define PORT 8080
#define BUFFER_SIZE 1024

char SHARED_FOLDER[] = "./";

void signal_handler(int sig) {
    while (waitpid(-1, NULL, WNOHANG) > 0);
}
```

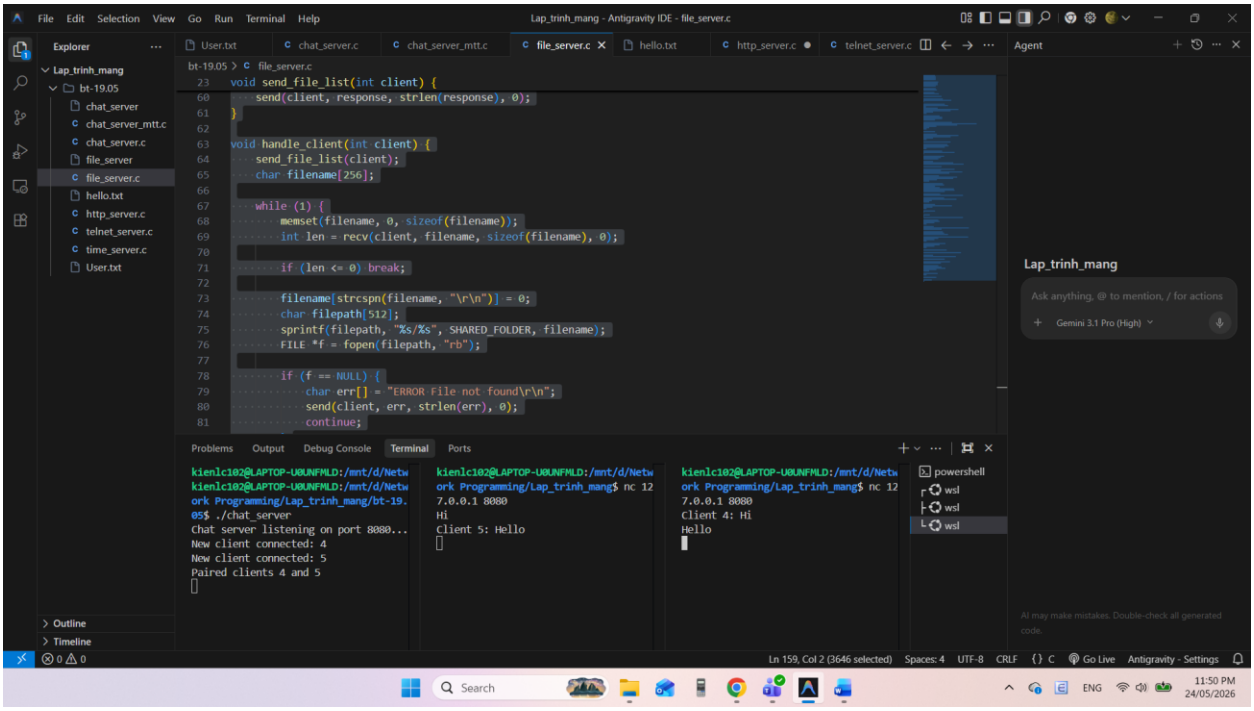
```
void send_file_list(int client) {
    DIR *dir = opendir(SHARED_FOLDER);

    if (dir == NULL) {
        char msg[] = "ERROR No files to download\r\n";
        send(client, msg, strlen(msg), 0);
        return;
    }

    struct dirent *entry;
    char response[4096] = "";
    char filenames[100][256];
    int count = 0;

    while ((entry = readdir(dir)) != NULL) {
        if (strcmp(entry->d_name, ".") != 0 && strcmp(entry->d_name, "..") != 0)
{
            strcpy(filenames[count], entry->d_name);
            count++;
        }
    }

    closedir(dir);

    if (count == 0) {
        char msg[] = "ERROR No files to download\r\n";
        send(client, msg, strlen(msg), 0);
        return;
    }

    sprintf(response, "OK %d\r\n", count);

    for (int i = 0; i < count; i++) {
        strcat(response, filenames[i]);
        strcat(response, "\r\n");
    }

    strcat(response, "\r\n");
    send(client, response, strlen(response), 0);
}

void handle_client(int client) {
    send_file_list(client);
    char filename[256];
```

```
    while (1) {
        memset(filename, 0, sizeof(filename));
        int len = recv(client, filename, sizeof(filename), 0);

        if (len <= 0) break;

        filename[strcspn(filename, "\r\n")] = 0;
        char filepath[512];
        sprintf(filepath, "%s/%s", SHARED_FOLDER, filename);
        FILE *f = fopen(filepath, "rb");

        if (f == NULL) {
            char err[] = "ERROR File not found\r\n";
            send(client, err, strlen(err), 0);
            continue;
        }

        fseek(f, 0, SEEK_END);
        int filesize = ftell(f);
        rewind(f);
        char header[128];
        sprintf(header, "OK %d\r\n", filesize);
        send(client, header, strlen(header), 0);
        char buffer[BUFFER_SIZE];

        while (!feof(f)) {
            int bytesRead = fread(buffer, 1, BUFFER_SIZE, f);

            if (bytesRead > 0) {
                send(client, buffer, bytesRead, 0);
            }
        }

        fclose(f);
        printf("Sent file: %s\n", filename);
        break;
    }

    close(client);
}

int main()
{
    int listener = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
```

```c
    if (listener < 0) {
        perror("socket");
        return 1;
    }

    int opt = 1;
    setsockopt(listener, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
    struct sockaddr_in addr;

    addr.sin_family = AF_INET;
    addr.sin_port = htons(PORT);
    addr.sin_addr.s_addr = INADDR_ANY;

    if (bind(listener, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        perror("bind");
        return 1;
    }

    if (listen(listener, 5) < 0) {
        perror("listen");
        return 1;
    }

    signal(SIGCHLD, signal_handler);

    printf("File server listening on port %d...\n", PORT);

    while (1) {
        int client = accept(listener, NULL, NULL);

        if (client < 0) continue;

        printf("New client connected\n");
        pid_t pid = fork();

        if (pid == 0) {
            close(listener);
            handle_client(client);
            exit(0);
        }

        close(client);
    }

    close(listener);
```

```
    return 0;
}
```

Bài 3.2 chat_server: ứng dụng chat đa luồng



Mã nguồn:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h>

#include <sys/socket.h>
#include <sys/types.h>
#include <arpa/inet.h>
#include <unistd.h>

#include <signal.h>
#include <sys/wait.h>

#define PORT 8080
#define BUFFER_SIZE 1024

char SHARED_FOLDER[] = "./";
```

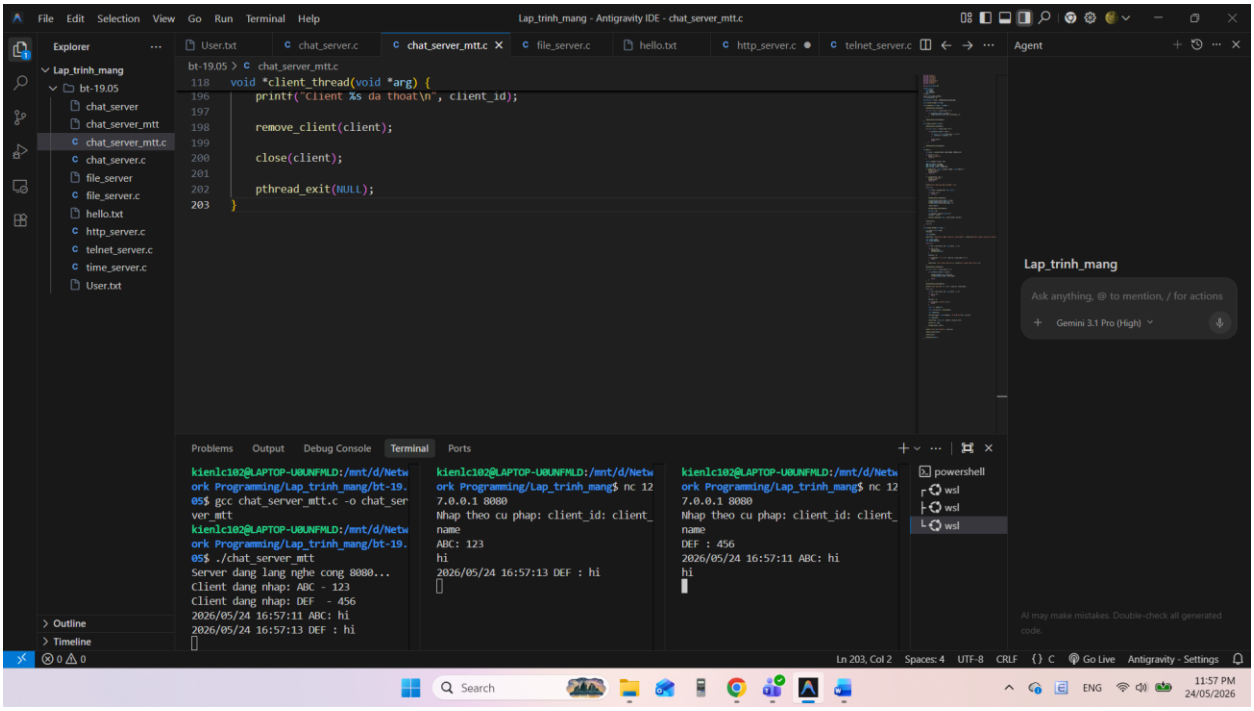
```c
void signal_handler(int sig) {
    while (waitpid(-1, NULL, WNOHANG) > 0);
}

void send_file_list(int client) {
    DIR *dir = opendir(SHARED_FOLDER);

    if (dir == NULL) {
        char msg[] = "ERROR No files to download\r\n";
        send(client, msg, strlen(msg), 0);
        return;
    }

    struct dirent *entry;
    char response[4096] = "";
    char filenames[100][256];
    int count = 0;

    while ((entry = readdir(dir)) != NULL) {
        if (strcmp(entry->d_name, ".") != 0 && strcmp(entry->d_name, "..") != 0)
{
            strcpy(filenames[count], entry->d_name);
            count++;
        }
    }

    closedir(dir);

    if (count == 0) {
        char msg[] = "ERROR No files to download\r\n";
        send(client, msg, strlen(msg), 0);
        return;
    }

    sprintf(response, "OK %d\r\n", count);

    for (int i = 0; i < count; i++) {
        strcat(response, filenames[i]);
        strcat(response, "\r\n");
    }

    strcat(response, "\r\n");
    send(client, response, strlen(response), 0);
}
```

```c
void handle_client(int client) {
    send_file_list(client);
    char filename[256];

    while (1) {
        memset(filename, 0, sizeof(filename));
        int len = recv(client, filename, sizeof(filename), 0);

        if (len <= 0) break;

        filename[strcspn(filename, "\r\n")] = 0;
        char filepath[512];
        sprintf(filepath, "%s/%s", SHARED_FOLDER, filename);
        FILE *f = fopen(filepath, "rb");

        if (f == NULL) {
            char err[] = "ERROR File not found\r\n";
            send(client, err, strlen(err), 0);
            continue;
        }

        fseek(f, 0, SEEK_END);
        int filesize = ftell(f);
        rewind(f);
        char header[128];
        sprintf(header, "OK %d\r\n", filesize);
        send(client, header, strlen(header), 0);
        char buffer[BUFFER_SIZE];

        while (!feof(f)) {
            int bytesRead = fread(buffer, 1, BUFFER_SIZE, f);

            if (bytesRead > 0) {
                send(client, buffer, bytesRead, 0);
            }
        }

        fclose(f);
        printf("Sent file: %s\n", filename);
        break;
    }

    close(client);
}
```

```c
int main()
{
    int listener = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

    if (listener < 0) {
        perror("socket");
        return 1;
    }

    int opt = 1;
    setsockopt(listener, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
    struct sockaddr_in addr;

    addr.sin_family = AF_INET;
    addr.sin_port = htons(PORT);
    addr.sin_addr.s_addr = INADDR_ANY;

    if (bind(listener, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        perror("bind");
        return 1;
    }

    if (listen(listener, 5) < 0) {
        perror("listen");
        return 1;
    }

    signal(SIGCHLD, signal_handler);

    printf("File server listening on port %d...\n", PORT);

    while (1) {
        int client = accept(listener, NULL, NULL);

        if (client < 0) continue;

        printf("New client connected\n");
        pid_t pid = fork();

        if (pid == 0) {
            close(listener);
            handle_client(client);
            exit(0);
        }
```

```
        close(client);
    }

    close(listener);

    return 0;
}
```

Bài chat_server (slide):



Mã nguồn:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <pthread.h>
#include <time.h>

#define MAX_CLIENTS 100
```

```c
typedef struct {
    int socket;
    char id[50];
    char name[50];
} Client;

Client clients[MAX_CLIENTS];
int client_count = 0;

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void *client_thread(void *arg);

void broadcast(char *msg, int sender) {

    pthread_mutex_lock(&mutex);

    for (int i = 0; i < client_count; i++) {

        if (clients[i].socket != sender) {
            send(clients[i].socket, msg, strlen(msg), 0);
        }
    }

    pthread_mutex_unlock(&mutex);
}

void remove_client(int sock) {

    pthread_mutex_lock(&mutex);

    for (int i = 0; i < client_count; i++) {

        if (clients[i].socket == sock) {

            for (int j = i; j < client_count - 1; j++) {
                clients[j] = clients[j + 1];
            }

            client_count--;
            break;
        }
    }

    pthread_mutex_unlock(&mutex);
```

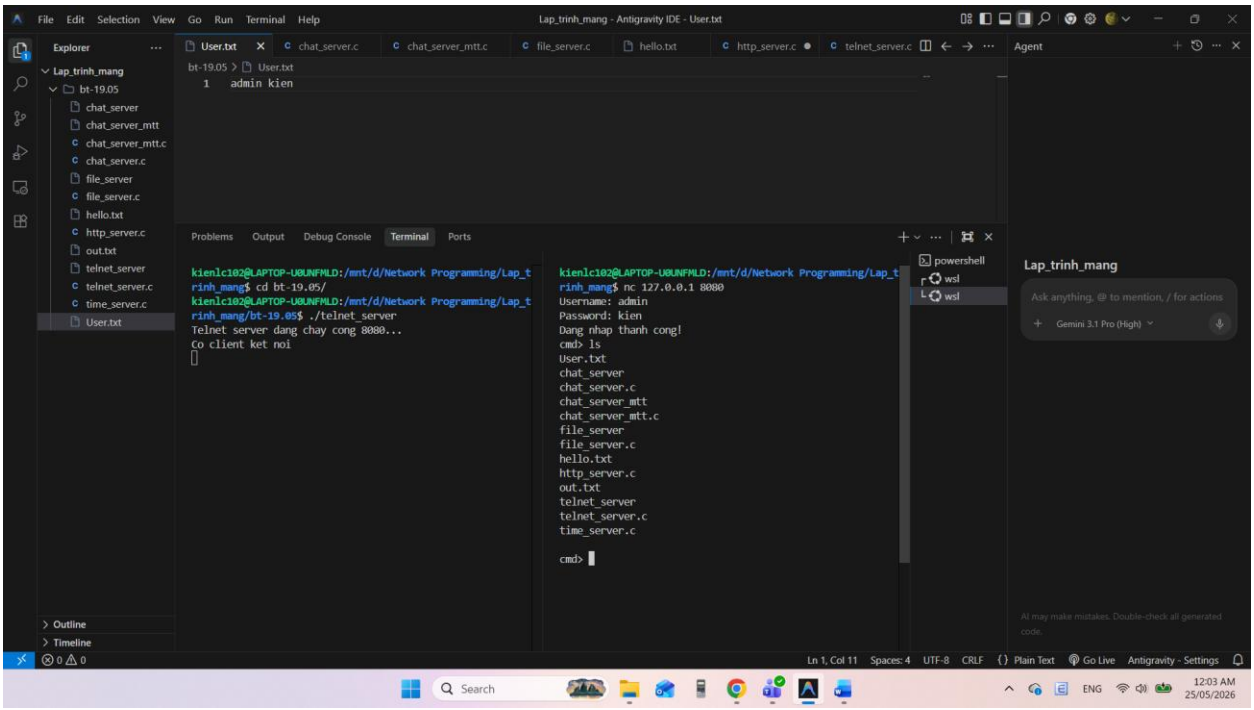
```
}

int main() {

    int server = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

    if (server == -1) {
        perror("socket()");
        return 1;
    }

    struct sockaddr_in addr = {0};

    addr.sin_family = AF_INET;
    addr.sin_port = htons(8080);
    addr.sin_addr.s_addr = INADDR_ANY;

    if (bind(server, (struct sockaddr *)&addr, sizeof(addr))) {
        perror("bind()");
        close(server);
        return 1;
    }

    if (listen(server, 10)) {
        perror("listen()");
        close(server);
        return 1;
    }

    printf("Server dang lang nghe cong 8080...\n");

    while (1) {

        int client = accept(server, NULL, NULL);

        if (client < 0) {
            continue;
        }

        pthread_mutex_lock(&mutex);

        clients[client_count].socket = client;
        strcpy(clients[client_count].id, "");
        strcpy(clients[client_count].name, "");
```

```
        client_count++;

        pthread_mutex_unlock(&mutex);

        pthread_t tid;

        int *pclient = malloc(sizeof(int));
        *pclient = client;

        pthread_create(&tid, NULL, client_thread, pclient);
    }

    close(server);

    return 0;
}

void *client_thread(void *arg) {

    int client = *(int *)arg;
    free(arg);

    char buf[1024];

    send(client, "Nhap theo cu phap: client_id: client_name\n",  strlen("Nhap
theo cu phap: client_id: client_name\n"), 0);

    char client_id[50];
    char client_name[50];

    while (1) {

        int len = recv(client, buf, sizeof(buf) - 1, 0);

        if (len <= 0) {
            close(client);
            pthread_exit(NULL);
        }

        buf[len] = 0;

        if (sscanf(buf, "%[^:]: %s", client_id, client_name) == 2) {
            break;
        }
```

```
        send(client, "Sai cu phap. Nhap lai!\n", strlen("Sai cu phap. Nhap
lai!\n"), 0);
    }

    pthread_mutex_lock(&mutex);

    for (int i = 0; i < client_count; i++) {

        if (clients[i].socket == client) {

            strcpy(clients[i].id, client_id);
            strcpy(clients[i].name, client_name);

            break;
        }
    }

    pthread_mutex_unlock(&mutex);

    printf("Client dang nhap: %s - %s\n", client_id, client_name);

    while (1) {

        int len = recv(client, buf, sizeof(buf) - 1, 0);

        if (len <= 0) {
            break;
        }

        buf[len] = 0;

        if (strcmp(buf, "exit\n") == 0) {
            break;
        }

        time_t t = time(NULL);

        struct tm *tm_info = localtime(&t);

        char timebuf[64];

        strftime(timebuf, sizeof(timebuf), "%Y/%m/%d %H:%M:%S", tm_info);

        char msg[1200];
```

```
        sprintf(msg, "%s %s: %s", timebuf, client_id, buf);

        printf("%s", msg);

        broadcast(msg, client);
    }

    printf("Client %s da thoat\n", client_id);

    remove_client(client);

    close(client);

    pthread_exit(NULL);
}
```

Bài telnet_server (slide):



Mã nguồn:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```c
#include <unistd.h>
#include <arpa/inet.h>
#include <pthread.h>

void *client_thread(void *arg);

int check_login(char *user, char *pass) {

    FILE *f = fopen("User.txt", "r");

    if (f == NULL) {
        return 0;
    }

    char db_user[100];
    char db_pass[100];

    while (fscanf(f, "%s %s", db_user, db_pass) != EOF) {

        if (strcmp(user, db_user) == 0 &&
            strcmp(pass, db_pass) == 0) {

            fclose(f);
            return 1;
        }
    }

    fclose(f);

    return 0;
}

int main() {

    int server = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

    if (server == -1) {
        perror("socket()");
        return 1;
    }

    struct sockaddr_in addr = {0};

    addr.sin_family = AF_INET;
    addr.sin_port = htons(8080);
```

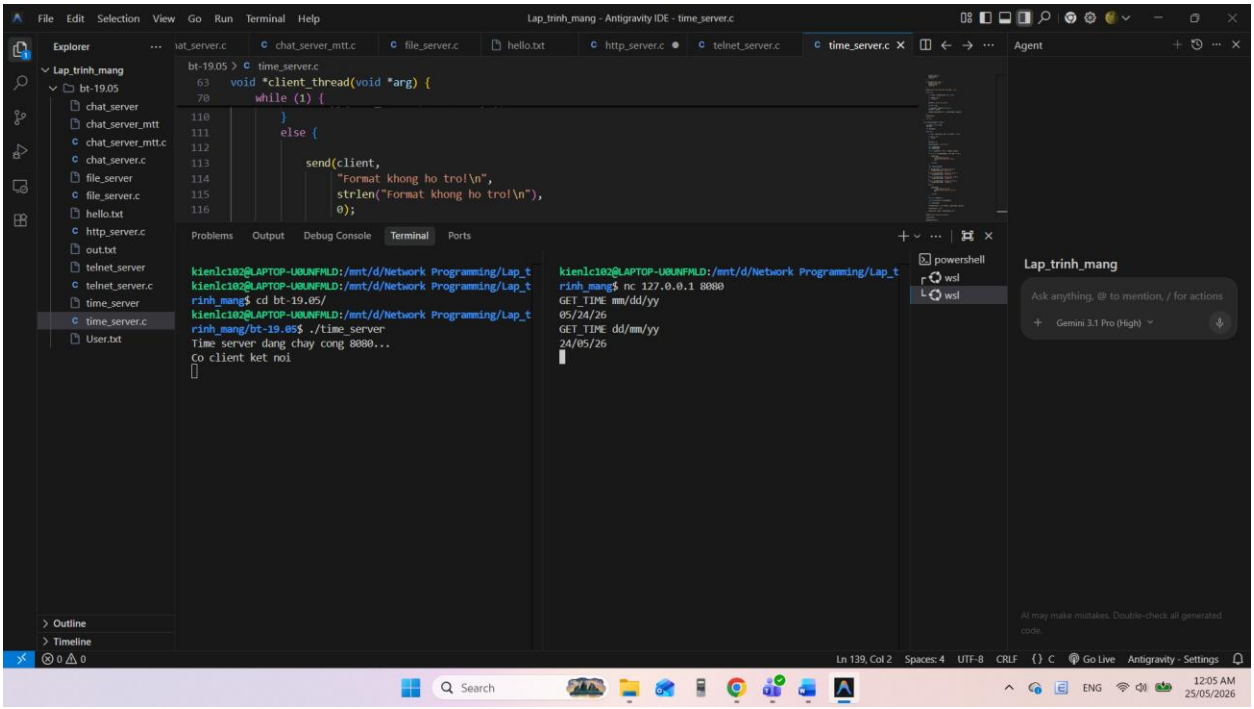
```c
    addr.sin_addr.s_addr = INADDR_ANY;

    if (bind(server, (struct sockaddr *)&addr, sizeof(addr))) {
        perror("bind()");
        close(server);
        return 1;
    }

    if (listen(server, 10)) {
        perror("listen()");
        close(server);
        return 1;
    }

    printf("Telnet server dang chay cong 8080...\n");

    while (1) {

        int client = accept(server, NULL, NULL);

        if (client < 0) {
            continue;
        }

        printf("Co client ket noi\n");

        pthread_t tid;

        int *pclient = malloc(sizeof(int));
        *pclient = client;

        pthread_create(&tid, NULL, client_thread, pclient);
    }

    close(server);

    return 0;
}

void *client_thread(void *arg) {

    int client = *(int *)arg;
    free(arg);

    char buf[4096];
```

```
    send(client, "Username: ", strlen("Username: "), 0);

    int len = recv(client, buf, sizeof(buf) - 1, 0);

    if (len <= 0) {
        close(client);
        pthread_exit(NULL);
    }

    buf[len] = 0;
    buf[strcspn(buf, "\r\n")] = 0;

    char user[100];
    strcpy(user, buf);

    send(client, "Password: ", strlen("Password: "), 0);

    len = recv(client, buf, sizeof(buf) - 1, 0);

    if (len <= 0) {
        close(client);
        pthread_exit(NULL);
    }

    buf[len] = 0;
    buf[strcspn(buf, "\r\n")] = 0;

    char pass[100];
    strcpy(pass, buf);

    if (!check_login(user, pass)) {

        send(client, "Dang nhap that bai!\n", strlen("Dang nhap that bai!\n"),
0);

        close(client);

        pthread_exit(NULL);
    }

    send(client, "Dang nhap thanh cong!\n", strlen("Dang nhap thanh cong!\n"),
0);

    while (1) {
```

```
        send(client, "cmd> ", strlen("cmd> "), 0);

        len = recv(client, buf, sizeof(buf) - 1, 0);

        if (len <= 0) {
            break;
        }

        buf[len] = 0;
        buf[strcspn(buf, "\r\n")] = 0;

        if (strcmp(buf, "exit") == 0) {
            break;
        }

        char command[5000];

        sprintf(command, "%s > out.txt", buf);

        system(command);

        FILE *f = fopen("out.txt", "r");

        if (f == NULL) {

            send(client, "Khong mo duoc file out.txt\n", strlen("Khong mo duoc
file out.txt\n"), 0);

            continue;
        }

        char output[4096];

        while (fgets(output, sizeof(output), f) != NULL) {

            send(client, output, strlen(output), 0);
        }

        fclose(f);

        send(client, "\n", 1, 0);
    }

    printf("Client da ngat ket noi\n");
```

```
    close(client);

    pthread_exit(NULL);
}
```

Bài time_server (slide):



Mã nguồn:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <pthread.h>
#include <time.h>

void *client_thread(void *arg);

int main() {

    int server = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
```

```c
if (server == -1) {
    perror("socket()");
    return 1;
}

struct sockaddr_in addr = {0};

addr.sin_family = AF_INET;
addr.sin_port = htons(8080);
addr.sin_addr.s_addr = INADDR_ANY;

if (bind(server, (struct sockaddr *)&addr, sizeof(addr))) {
    perror("bind()");
    close(server);
    return 1;
}

if (listen(server, 10)) {
    perror("listen()");
    close(server);
    return 1;
}

printf("Time server dang chay cong 8080...\n");

while (1) {

    int client = accept(server, NULL, NULL);

    if (client < 0) {
        continue;
    }

    printf("Co client ket noi\n");

    pthread_t tid;

    int *pclient = malloc(sizeof(int));
    *pclient = client;

    pthread_create(&tid, NULL, client_thread, pclient);
}

close(server);
```

```c
    return 0;
}

void *client_thread(void *arg) {

    int client = *(int *)arg;
    free(arg);

    char buf[1024];

    while (1) {

        int len = recv(client, buf, sizeof(buf) - 1, 0);

        if (len <= 0) {
            break;
        }

        buf[len] = 0;

        buf[strcspn(buf, "\r\n")] = 0;

        char command[100];
        char format[100];

        int n = sscanf(buf, "%s %s", command, format);

        if (n != 2 || strcmp(command, "GET_TIME") != 0) {

            send(client,
                 "Lenh khong hop le!\n",
                 strlen("Lenh khong hop le!\n"),
                 0);

            continue;
        }

        char time_format[50];

        if (strcmp(format, "dd/mm/yyyy") == 0) {
            strcpy(time_format, "%d/%m/%Y");
        }
        else if (strcmp(format, "dd/mm/yy") == 0) {
            strcpy(time_format, "%d/%m/%y");
```

```
        }
        else if (strcmp(format, "mm/dd/yyyy") == 0) {
            strcpy(time_format, "%m/%d/%Y");
        }
        else if (strcmp(format, "mm/dd/yy") == 0) {
            strcpy(time_format, "%m/%d/%y");
        }
        else {

            send(client,
                 "Format khong ho tro!\n",
                 strlen("Format khong ho tro!\n"),
                 0);

            continue;
        }

        time_t t = time(NULL);

        struct tm *tm_info = localtime(&t);

        char result[100];

        strftime(result, sizeof(result), time_format, tm_info);

        strcat(result, "\n");

        send(client, result, strlen(result), 0);
    }

    printf("Client da ngat ket noi\n");

    close(client);

    pthread_exit(NULL);
}
```

Bài http_server (slide):

Mã nguồn:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <pthread.h>

#define THREAD_COUNT 5

int listener;

void *worker_thread(void *arg);

int main() {

    listener = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

    if (listener == -1) {
        perror("socket()");
        return 1;
    }
```

```
    struct sockaddr_in addr = {0};

    addr.sin_family = AF_INET;
    addr.sin_port = htons(8080);
    addr.sin_addr.s_addr = INADDR_ANY;

    if (bind(listener, (struct sockaddr *)&addr, sizeof(addr))) {
        perror("bind()");
        close(listener);
        return 1;
    }

    if (listen(listener, 10)) {
        perror("listen()");
        close(listener);
        return 1;
    }

    printf("HTTP Server dang chay cong 8080...\n");

    pthread_t tids[THREAD_COUNT];

    for (int i = 0; i < THREAD_COUNT; i++) {

        pthread_create(&tids[i], NULL, worker_thread, NULL);
    }

    for (int i = 0; i < THREAD_COUNT; i++) {

        pthread_join(tids[i], NULL);
    }

    close(listener);

    return 0;
}

void *worker_thread(void *arg) {

    while (1) {

        int client = accept(listener, NULL, NULL);

        if (client < 0) {
            continue;
```

```
        }

        printf("New client connected: %d\n", client);

        char buf[2048];

        int ret = recv(client, buf, sizeof(buf) - 1, 0);

        if (ret <= 0) {
            close(client);
            continue;
        }

        buf[ret] = 0;

        puts(buf);

        char *msg =
        "HTTP/1.1 200 OK\r\n"
        "Content-Type: text/html\r\n"
        "\r\n"
        "<html>"
        "<body>"
        "<h1>Xin chao cac ban</h1>"
        "</body>"
        "</html>";

        send(client, msg, strlen(msg), 0);

        close(client);
    }

    pthread_exit(NULL);
}
```